Data Preparation

```
!pip install lightgbm
```

```
Collecting lightgbm
  Downloading lightgbm-4.6.0-py3-none-manylinux_2_28_x86_64.whl.metadata (17 kB)
Requirement already satisfied: numpy>=1.17.0 in /usr/local/lib/python3.11/dist-packages (from lightgbm) (2.0.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.11/dist-packages (from lightgbm) (1.15.3)
Downloading lightgbm-4.6.0-py3-none-manylinux_2_28_x86_64.whl (3.6 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.6/3.6 MB 25.4 MB/s eta 0:00:00
Installing collected packages: lightgbm
Successfully installed lightgbm-4.6.0
```

```
import pandas as pd
import numpy as np
import lightgbm as lgb
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.metrics import ConfusionMatrixDisplay
import matplotlib.pyplot as plt
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
df = pd.read_csv('/content/drive/MyDrive/100_Batches_IndPenSim_V3.csv', low_memory=False)
```

```
df = df.dropna(subset=['Batch ID'])
```

```
df = df.sample(n=2000, random_state=42)
```

```
top_batches = df['Batch ID'].value_counts().nlargest(10).index
```

```
df = df[df['Batch ID'].isin(top_batches)]
```

```
X = df.select_dtypes(include=[np.number]).drop(columns=['Batch ID'], errors='ignore')
y = df['Batch ID']
```

Data Validation

```
print("Class distribution:\n", y.value_counts())
```

```
Class distribution:
 Batch ID
0.0         19
21292.0      3
112600.0     3
137890.0     2
107140.0     2
118750.0     2
120910.0     2
75093.0      2
150240.0     2
128200.0     2
Name: count, dtype: int64
```

```
print("Missing values:\n", X.isnull().sum())
```

```
Missing values:
 Time (h)                    0
Aeration rate(Fg:L/h)        0
Agitator RPM(RPM:RPM)        0
Sugar feed rate(Fs:L/h)      0
Acid flow rate(Fa:L/h)       0
                            ..
205                          0
204                          0
203                          0
202                         39
201                         39
Length: 2238, dtype: int64
```

Data Splitting & Scaling & Model Training

```
print(df.columns)
```

```
Index(['Time (h)', 'Aeration rate(Fg:L/h)', 'Agitator RPM(RPM:RPM)',
       'Sugar feed rate(Fs:L/h)', 'Acid flow rate(Fa:L/h)',
       'Base flow rate(Fb:L/h)', 'Heating/cooling water flow rate(Fc:L/h)',
       'Heating water flow rate(Fh:L/h)',
       'Water for injection/dilution(Fw:L/h)',
       'Air head pressure(pressure:bar)',
       ...
       '210', '209', '208', '207', '206', '205', '204', '203', '202', '201'],
      dtype='object', length=2239)
```

```
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
scaler = StandardScaler()
X_train_scaled, X_test_scaled, y_train, y_test = train_test_split(X, y, test_size=0.4, stratify=y, random_state=42)
X_train_scaled = scaler.fit_transform(X_train_scaled)
X_test_scaled = scaler.transform(X_test_scaled)
```

```
/usr/local/lib/python3.11/dist-packages/sklearn/utils/extmath.py:1101: RuntimeWarning: invalid value encountered in divide
  updated_mean = (last_sum + new_sum) / updated_sample_count
/usr/local/lib/python3.11/dist-packages/sklearn/utils/extmath.py:1106: RuntimeWarning: invalid value encountered in divide
  T = new_sum / new_sample_count
/usr/local/lib/python3.11/dist-packages/sklearn/utils/extmath.py:1126: RuntimeWarning: invalid value encountered in divide
  new_unnormalized_variance -= correction**2 / new_sample_count
```

```
model = RandomForestClassifier(random_state=42)
```

```
param_grid = {
    'n_estimators': [50, 100],
    'max_depth': [6, 10, 15]
}
```

```
grid_search = GridSearchCV(estimator=model, param_grid=param_grid, cv=3, scoring='accuracy', verbose=1)
```

```
grid_search.fit(X_train_scaled, y_train)
```

```
/usr/local/lib/python3.11/dist-packages/sklearn/model_selection/_split.py:805: UserWarning: The least populated class in y has only
  warnings.warn(
Fitting 3 folds for each of 6 candidates, totalling 18 fits
```

GridSearchCV

best_estimator_: RandomForestClassifier

RandomForestClassifier

```
best_params = grid_search.best_params_
best_score = grid_search.best_score_
```

```
print(f"Best Parameters: {best_params}")
print(f"Best Cross-Validation Accuracy: {best_score:.2f}")
```

```
Best Parameters: {'max_depth': 6, 'n_estimators': 50}
Best Cross-Validation Accuracy: 0.70
```

```
best_model = RandomForestClassifier(**best_params, random_state=42)
best_model.fit(X_train_scaled, y_train)
```

RandomForestClassifier

```
RandomForestClassifier(max_depth=6, n_estimators=50, random_state=42)
```

Evaluation

```
y_pred = best_model.predict(X_test_scaled)
```

```
accuracy = accuracy_score(y_test, y_pred)
print(f"Test Accuracy: {accuracy:.2f}")
```

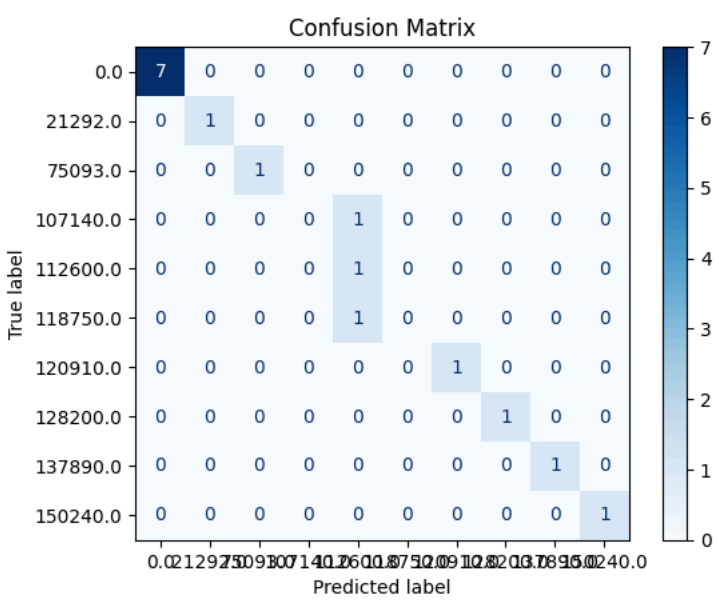
```
Test Accuracy: 0.88
```

```
print("Classification Report:")
print(classification_report(y_test, y_pred))
```

```
Classification Report:
               precision    recall  f1-score   support

          0.0       1.00      1.00      1.00         7
      21292.0       1.00      1.00      1.00         1
      75093.0       1.00      1.00      1.00         1
     107140.0       0.00      0.00      0.00         1
     112600.0       0.33      1.00      0.50         1
     118750.0       0.00      0.00      0.00         1
     120910.0       1.00      1.00      1.00         1
     128200.0       1.00      1.00      1.00         1
     137890.0       1.00      1.00      1.00         1
     150240.0       1.00      1.00      1.00         1

     accuracy                           0.88        16
    macro avg       0.73      0.80      0.75        16
 weighted avg       0.83      0.88      0.84        16

/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined ar
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined ar
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined ar
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=best_model.classes_)
disp.plot(cmap='Blues', ax=plt.gca())
plt.title("Confusion Matrix")
plt.show()
```



Error Handling

```
X.columns = [col.replace(' ', '_').replace('-', '_').replace('.', '_') for col in X.columns]
```

```
class_distribution = y.value_counts()
print("Class distribution:\n", class_distribution)
```

```
Class distribution:
 Batch ID
0.0         19
21292.0      3
112600.0     3
137890.0     2
107140.0     2
118750.0     2
120910.0     2
75093.0      2
```

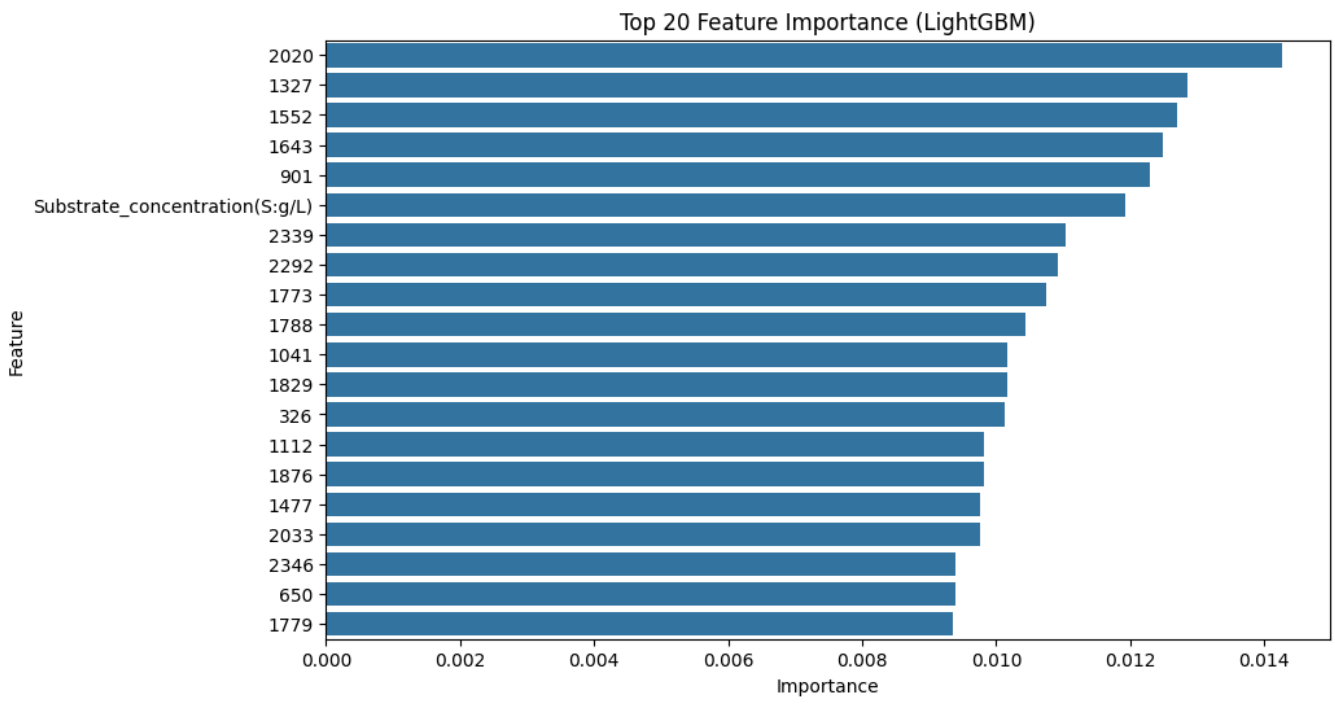
```
150240.0     2
128200.0     2
Name: count, dtype: int64
```

```
min_samples_required = 5  # Define a threshold for minimum samples per class
classes_with_few_samples = class_distribution[class_distribution < min_samples_required]
if not classes_with_few_samples.empty:
    print("Warning: The following classes have fewer than the minimum required samples:")
    print(classes_with_few_samples)
else:
    print("All classes have sufficient samples.")
```

```
Warning: The following classes have fewer than the minimum required samples:
Batch ID
21292.0     3
112600.0    3
137890.0    2
107140.0    2
118750.0    2
120910.0    2
75093.0     2
150240.0    2
128200.0    2
Name: count, dtype: int64
```

Feature Importance or Model Interpretability

```
import seaborn as sns
importance_df = pd.DataFrame({
    'Feature': X.columns,
    'Importance': best_model.feature_importances_
}).sort_values(by='Importance', ascending=False)

top_features = importance_df.head(20)  # Show only top 20 important features
plt.figure(figsize=(10, 6))
sns.barplot(x='Importance', y='Feature', data=top_features)
plt.title("Top 20 Feature Importance (LightGBM)")
plt.show()
```



Testing Additional Models for Comparison

```
!pip install xgboost
```

```
Collecting xgboost
  Downloading xgboost-3.0.2-py3-none-manylinux_2_28_x86_64.whl.metadata (2.1 kB)
Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages (from xgboost) (2.0.2)
Collecting nvidia-nccl-cu12 (from xgboost)
  Downloading nvidia_nccl_cu12-2.26.5-py3-none-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (2.0 kB)
Requirement already satisfied: scipy in /usr/local/lib/python3.11/dist-packages (from xgboost) (1.15.3)
```

```
Downloading xgboost-3.0.2-py3-none-manylinux_2_28_x86_64.whl (253.9 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 253.9/253.9 MB 4.3 MB/s eta 0:00:00
Downloading nvidia_nccl_cu12-2.26.5-py3-none-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (318.1 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 318.1/318.1 MB 3.4 MB/s eta 0:00:00
Installing collected packages: nvidia-nccl-cu12, xgboost
Successfully installed nvidia-nccl-cu12-2.26.5 xgboost-3.0.2
```

```
import xgboost as xgb


rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train_scaled, y_train)
y_pred_rf = rf_model.predict(X_test_scaled)
print(f"Random Forest Accuracy: {accuracy_score(y_test, y_pred_rf) * 100:.2f}%")
```

```
Random Forest Accuracy: 87.50%
```

```
from sklearn.preprocessing import LabelEncoder


encoder = LabelEncoder()
y_train_encoded = encoder.fit_transform(y_train)
y_test_encoded = encoder.transform(y_test)


xgb_model = xgb.XGBClassifier(n_estimators=100, learning_rate=0.1, random_state=42)
xgb_model.fit(X_train_scaled, y_train_encoded)
y_pred_xgb = xgb_model.predict(X_test_scaled)


y_pred_xgb_decoded = encoder.inverse_transform(y_pred_xgb)
print(f"XGBoost Accuracy: {accuracy_score(y_test_encoded, y_pred_xgb) * 100:.2f}%")
```

```
XGBoost Accuracy: 62.50%
```

Advanced Hyperparameter Optimization

```
!pip install bayesian-optimization
from bayes_opt import BayesianOptimization
import lightgbm as lgb
from sklearn.metrics import accuracy_score
```

```
Collecting bayesian-optimization
  Downloading bayesian_optimization-2.0.4-py3-none-any.whl.metadata (9.0 kB)
Collecting colorama<0.5.0,>=0.4.6 (from bayesian-optimization)
  Downloading colorama-0.4.6-py2.py3-none-any.whl.metadata (17 kB)
Requirement already satisfied: numpy>=1.25 in /usr/local/lib/python3.11/dist-packages (from bayesian-optimization) (2.0.2)
Requirement already satisfied: scikit-learn<2.0.0,>=1.0.0 in /usr/local/lib/python3.11/dist-packages (from bayesian-optimization) (1
Requirement already satisfied: scipy<2.0.0,>=1.0.0 in /usr/local/lib/python3.11/dist-packages (from bayesian-optimization) (1.15.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn<2.0.0,>=1.0.0->bayesian-o
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn<2.0.0,>=1.0.0->bay
Downloading bayesian_optimization-2.0.4-py3-none-any.whl (31 kB)
Downloading colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Installing collected packages: colorama, bayesian-optimization
Successfully installed bayesian-optimization-2.0.4 colorama-0.4.6
```

```
def lgb_eval(n_estimators, max_depth, learning_rate, num_leaves):
    params = {
        'n_estimators': int(n_estimators),
        'max_depth': int(max_depth),
        'learning_rate': learning_rate,
        'num_leaves': int(num_leaves)
    }
    model = lgb.LGBMClassifier(**params, random_state=42)
    model.fit(X_train, y_train_encoded)  # Ensure y is properly encoded!
    return accuracy_score(y_test_encoded, model.predict(X_test))  # Return accuracy


optimizer = BayesianOptimization(
    f=lgb_eval,
    pbounds={
        'n_estimators': (50, 500),
        'max_depth': (3, 15),
        'learning_rate': (0.01, 0.1),
        'num_leaves': (20, 150)
    },
    random_state=42
)
```

Anomaly Detection

```
from sklearn.ensemble import IsolationForest


X = df.select_dtypes(include=[np.number]).drop(columns=['Batch ID'], errors='ignore')


iso_forest = IsolationForest(contamination=0.05, random_state=42)
df['Anomaly_Score'] = iso_forest.fit_predict(X)


anomalies = df[df['Anomaly_Score'] == -1]


print(f"Total Anomalies Detected: {len(anomalies)}")
print(anomalies.head())
```

```
Total Anomalies Detected: 2
       Time (h)  Aeration rate(Fg:L/h)  Agitator RPM(RPM:RPM)  \
43607      20.6                     55                    100
83722      78.6                     60                    100

       Sugar feed rate(Fs:L/h)  Acid flow rate(Fa:L/h)  \
43607                      150                     0.0
83722                      116                     0.0

       Base flow rate(Fb:L/h)  Heating/cooling water flow rate(Fc:L/h)  \
43607                 106.980                                  381.77
83722                  90.859                                  130.63

       Heating water flow rate(Fh:L/h)  Water for injection/dilution(Fw:L/h)  \
43607                           0.0001                                     0
83722                           0.0001                                   100

       Air head pressure(pressure:bar)  ...        209        208        207  \
43607                              0.7  ...   523140.0   529530.0   534590.0
83722                              1.1  ...  1208600.0  1215400.0  1220800.0

             206        205        204        203  202  201  Anomaly_Score
43607   537670.0   538910.0   538290.0   536410.0  NaN  NaN             -1
83722  1224400.0  1226100.0  1226100.0  1225000.0  NaN  NaN             -1

[2 rows x 2240 columns]
```

Cluster Analysis

```
from sklearn.cluster import KMeans


X_cleaned = X.dropna()


from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy='mean')
X_imputed = imputer.fit_transform(X)
```

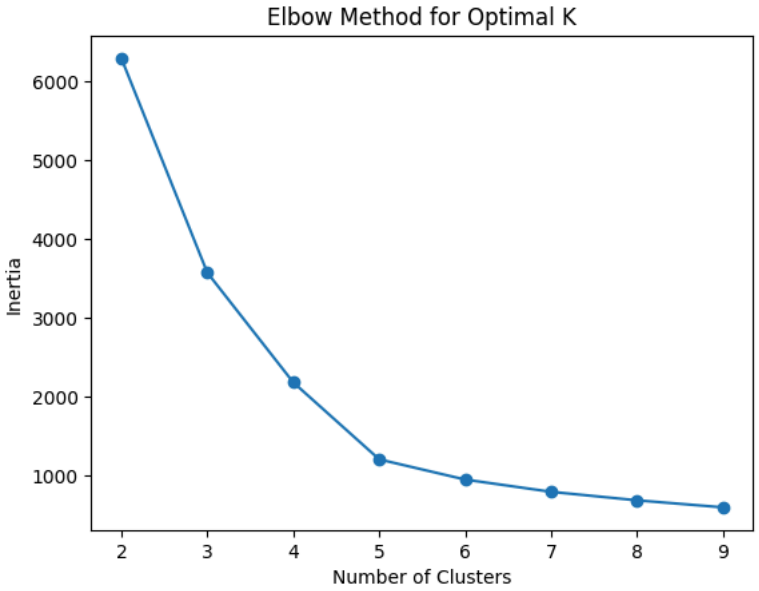
```
/usr/local/lib/python3.11/dist-packages/sklearn/impute/_base.py:635: UserWarning: Skipping features without any observed values: ['2
  warnings.warn(
```

```
print("Missing values:\n", X.isnull().sum().sum())
```

```
Missing values:
 258
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_imputed)


inertia = []
for k in range(2, 10):
    model = KMeans(n_clusters=k, random_state=42, n_init=10)
    model.fit(X_scaled)
    inertia.append(model.inertia_)
```

```
plt.plot(range(2, 10), inertia, marker='o')
plt.xlabel('Number of Clusters')
plt.ylabel('Inertia')
plt.title('Elbow Method for Optimal K')
plt.show()
```



Display Input Data

```
X = df.select_dtypes(include=[np.number]).drop(columns=['Batch ID'], errors='ignore')
y = df['Batch ID']
```

```
print(f"Input Features Shape: {X.shape}")
print(f"Target Labels Shape: {y.shape}")
```

```
Input Features Shape: (39, 2239)
Target Labels Shape: (39,)
```

```
print("\nSample Input Data:")
print(X.head())
```

```
Sample Input Data:
        Time (h)  Aeration rate(Fg:L/h)  Agitator RPM(RPM:RPM)  \
66548       1.8                     30                    100
43476     224.4                     65                    100
74523       0.8                     30                    100
109263      1.8                     30                    100
36238     132.8                     75                    100

        Sugar feed rate(Fs:L/h)  Acid flow rate(Fa:L/h)  \
66548                         8                     0.0
43476                        45                     0.0
74523                         8                     0.0
109263                        8                     0.0
36238                       105                     0.0

        Base flow rate(Fb:L/h)  Heating/cooling water flow rate(Fc:L/h)  \
66548                  26.2470                                 52.7450
43476                   6.5544                                  2.0604
74523                 160.9200                                  0.0001
109263                 18.4370                                 36.1400
36238                 149.5800                                  7.6862

        Heating water flow rate(Fh:L/h)  Water for injection/dilution(Fw:L/h)  \
66548                           4.36850                                     0
43476                           0.00010                                   250
74523                         103.43000                                     0
109263                          0.59694                                     0
36238                           0.00010                                   100

        Air head pressure(pressure:bar)  ...         209         208         207  \
66548                               0.6  ...         0.0         0.0         0.0
43476                               0.9  ...   2103700.0   2111700.0   2118300.0
74523                               0.6  ...         0.0         0.0         0.0
109263                              0.6  ...         0.0         0.0         0.0
36238                               1.0  ...   1842400.0   1849900.0   1856100.0
```

```
                  206        205        204        203  202  201  Anomaly_Score
66548         0.0        0.0        0.0        0.0  NaN  NaN              1
43476   2122800.0  2125200.0  2126100.0  2126500.0  NaN  NaN              1
74523         0.0        0.0        0.0        0.0  NaN  NaN              1
109263        0.0        0.0        0.0        0.0  NaN  NaN              1
36238   1860200.0  1862500.0  1863300.0  1863200.0  NaN  NaN              1

[5 rows x 2239 columns]
```

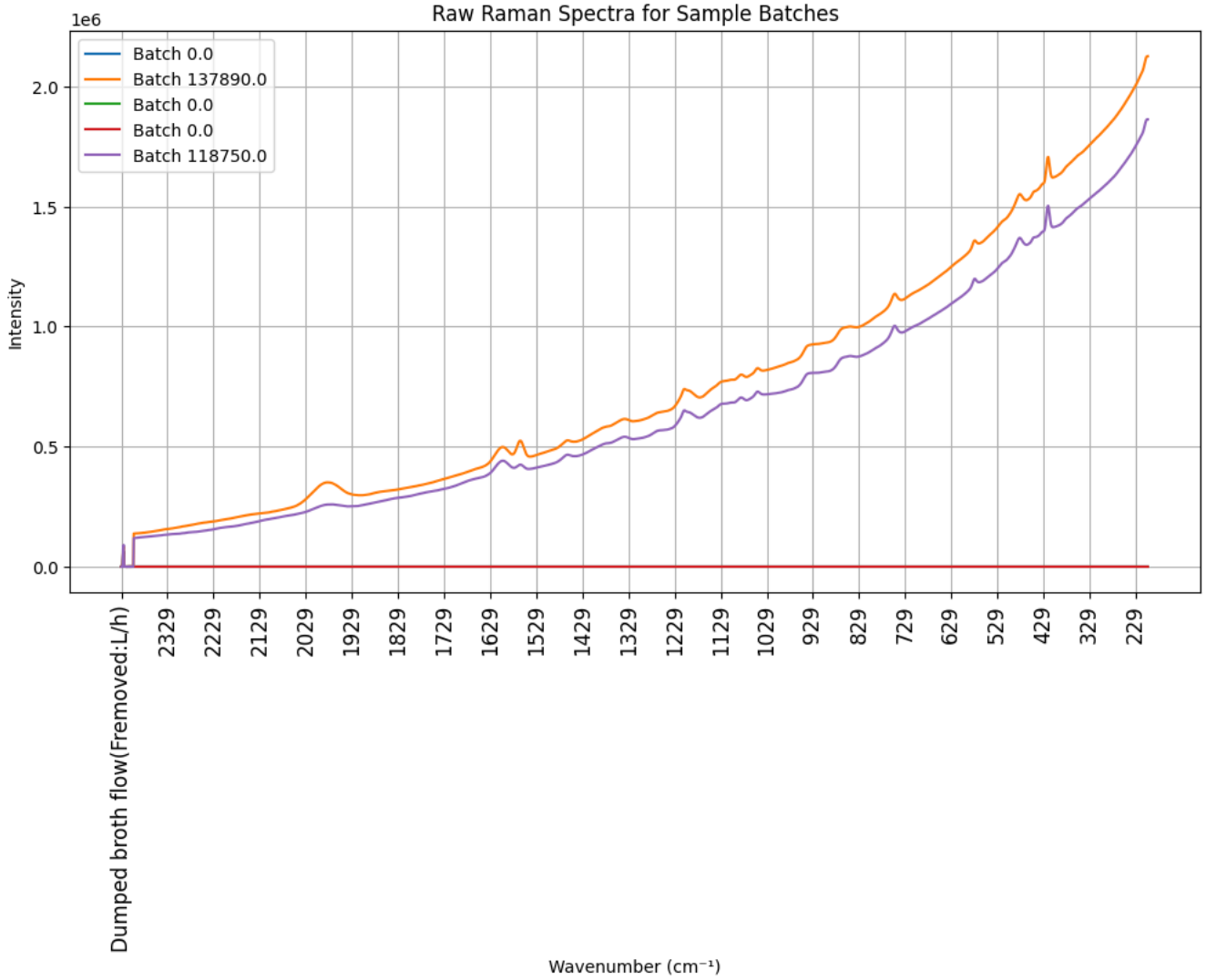
```
print("Available Columns:\n", df.columns)
```

```
Available Columns:
 Index(['Time (h)', 'Aeration rate(Fg:L/h)', 'Agitator RPM(RPM:RPM)',
       'Sugar feed rate(Fs:L/h)', 'Acid flow rate(Fa:L/h)',
       'Base flow rate(Fb:L/h)', 'Heating/cooling water flow rate(Fc:L/h)',
       'Heating water flow rate(Fh:L/h)',
       'Water for injection/dilution(Fw:L/h)',
       'Air head pressure(pressure:bar)',
       ...
       '209', '208', '207', '206', '205', '204', '203', '202', '201',
       'Anomaly_Score'],
      dtype='object', length=2240)
```

Plotting The Raw Spectral Data

```
spectral_columns = df.columns[10:]
spectra_data = df[spectral_columns]
```

```
plt.figure(figsize=(12, 6))
for i in range(5):
    plt.plot(spectral_columns, spectra_data.iloc[i], label=f'Batch {df["Batch ID"].iloc[i]}')
plt.xticks(ticks=spectral_columns[::100], rotation=90, fontsize=12)
plt.xlabel("Wavenumber (cm⁻¹)")
plt.ylabel("Intensity")
plt.title("Raw Raman Spectra for Sample Batches")
plt.legend()
plt.grid(True)
plt.show()
```



Identification Of Fingerprint Region

```
fingerprint_start, fingerprint_end = 400, 1800

fingerprint_columns = [col for col in spectral_columns if col.replace('.', '').isdigit() and fingerprint_start <= float(col) <= fingerpr
fingerprint_spectra = spectra_data[fingerprint_columns]

print(f"Filtered Fingerprint Columns:\n{fingerprint_columns[:5]} ... {fingerprint_columns[-5:]}")
```

```
Filtered Fingerprint Columns:
['1800', '1799', '1798', '1797', '1796'] ... ['404', '403', '402', '401', '400']
```

Cut Spectra

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
fingerprint_scaled = scaler.fit_transform(fingerprint_spectra)

print(f"Preprocessed Spectral Shape: {fingerprint_scaled.shape}")
```

```
Preprocessed Spectral Shape: (39, 1401)
```

Extract Fingerprint Region & Cut Spectra

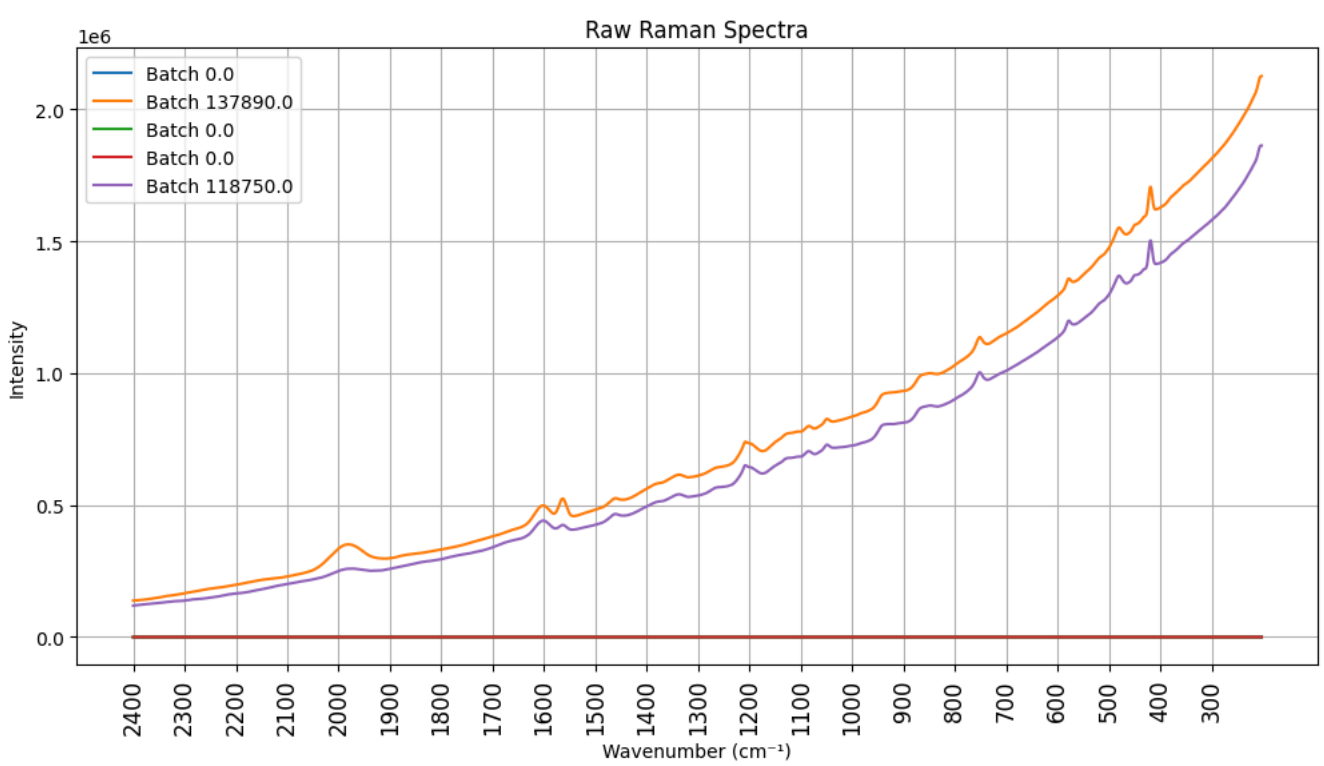
```
fingerprint_start, fingerprint_end = 400, 1800
```

```
spectral_columns = [col for col in df.columns if col.replace('.', '').isdigit()]


fingerprint_columns = [col for col in spectral_columns if fingerprint_start <= float(col) <= fingerprint_end]
fingerprint_spectra = df[fingerprint_columns]


print(f"Selected Fingerprint Region: {fingerprint_columns[:5]} ... {fingerprint_columns[-5:]}")
```

```
Selected Fingerprint Region: ['1800', '1799', '1798', '1797', '1796'] ... ['404', '403', '402', '401', '400']
```

Plotting Raw Spectral Data

```
import matplotlib.pyplot as plt

plt.figure(figsize=(12, 6))
for i in range(5):
    plt.plot(spectral_columns, df[spectral_columns].iloc[i], label=f'Batch {df["Batch ID"].iloc[i]}')
plt.xticks(ticks=spectral_columns[::100], rotation=90, fontsize=12)
plt.xlabel("Wavenumber (cm⁻¹)")
plt.ylabel("Intensity")
plt.title("Raw Raman Spectra")
plt.legend()
plt.grid(True)
plt.show()
```



Fingerprint Region From Literature

```
literature_fingerprint_start, literature_fingerprint_end = 1550, 1900


spectral_columns = [col for col in df.columns if col.replace('.', '').isdigit()]
literature_fingerprint_columns = [col for col in spectral_columns if literature_fingerprint_start <= float(col) <= literature_fingerpri


literature_fingerprint_spectra = df[literature_fingerprint_columns]
print(f"Extracted Fingerprint Columns:\n{literature_fingerprint_columns[:5]} ... {literature_fingerprint_columns[-5:]}")
```

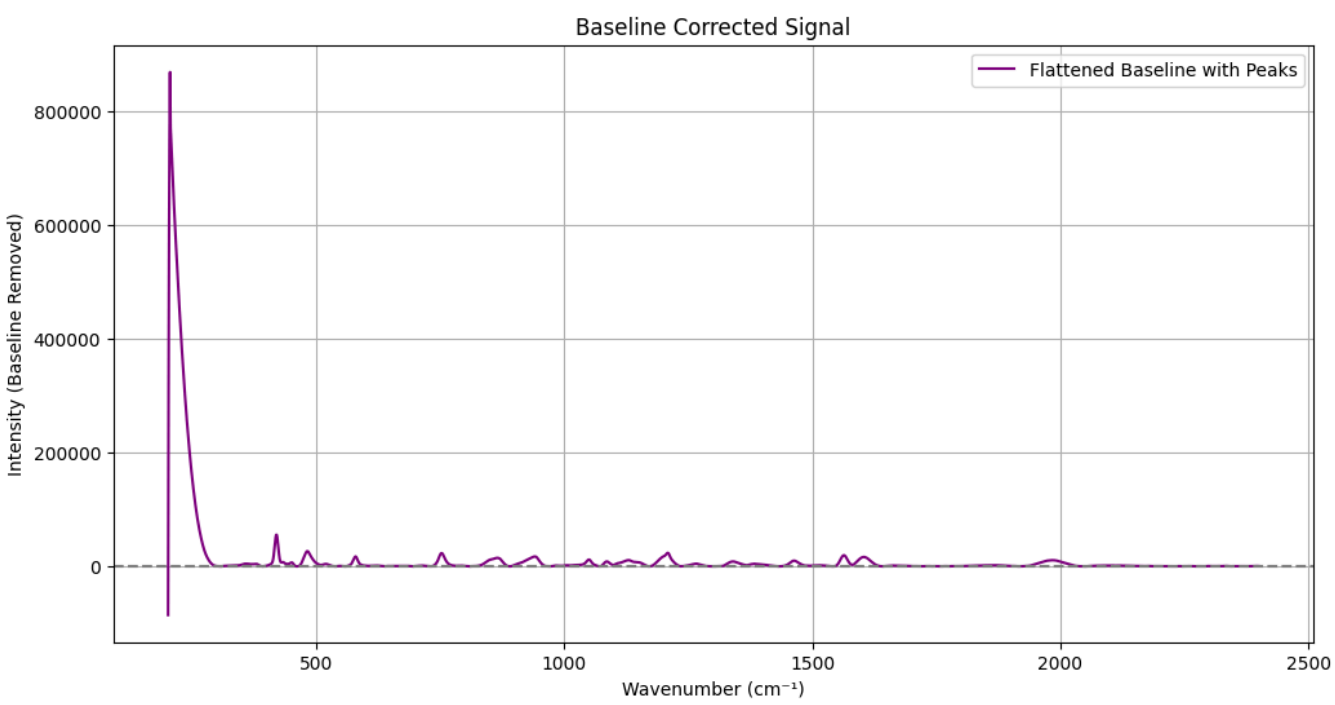
```
Extracted Fingerprint Columns:
['1900', '1899', '1898', '1897', '1896'] ... ['1554', '1553', '1552', '1551', '1550']
```

Baseline Correction

```
!pip install pybaselines
```

```
Collecting pybaselines
  Downloading pybaselines-1.2.0-py3-none-any.whl.metadata (9.9 kB)
Requirement already satisfied: numpy>=1.20 in /usr/local/lib/python3.11/dist-packages (from pybaselines) (2.0.2)
Requirement already satisfied: scipy>=1.6 in /usr/local/lib/python3.11/dist-packages (from pybaselines) (1.15.3)
Downloading pybaselines-1.2.0-py3-none-any.whl (209 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 210.0/210.0 kB 4.1 MB/s eta 0:00:00
Installing collected packages: pybaselines
Successfully installed pybaselines-1.2.0
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from pybaselines import Baseline
from scipy.signal import savgol_filter


spectral_columns = [col for col in df.columns if col.replace('.', '').isdigit()]
wn = np.array([float(col) for col in spectral_columns])
y_data = df[spectral_columns].values.mean(axis=0)

y_data_cleaned = np.nan_to_num(y_data, nan=0.0, posinf=0.0, neginf=0.0)

baseline_fitter = Baseline(wn)
baseline, _ = baseline_fitter.asls(y_data_cleaned, lam=1e2, p=0.0001)

flattened_spectrum = y_data_cleaned - baseline

smooth_spectrum = savgol_filter(flattened_spectrum, window_length=7, polyorder=2)

plt.figure(figsize=(12, 6))
plt.plot(wn, smooth_spectrum, label='Flattened Baseline with Peaks', color='purple')
plt.axhline(y=0, color='gray', linestyle='dashed')  # Flat reference line
plt.xlabel("Wavenumber (cm⁻¹)")
plt.ylabel("Intensity (Baseline Removed)")
plt.title("Baseline Corrected Signal")
plt.legend()
plt.grid(True)
plt.show()
```



Reading Literature from Dataset

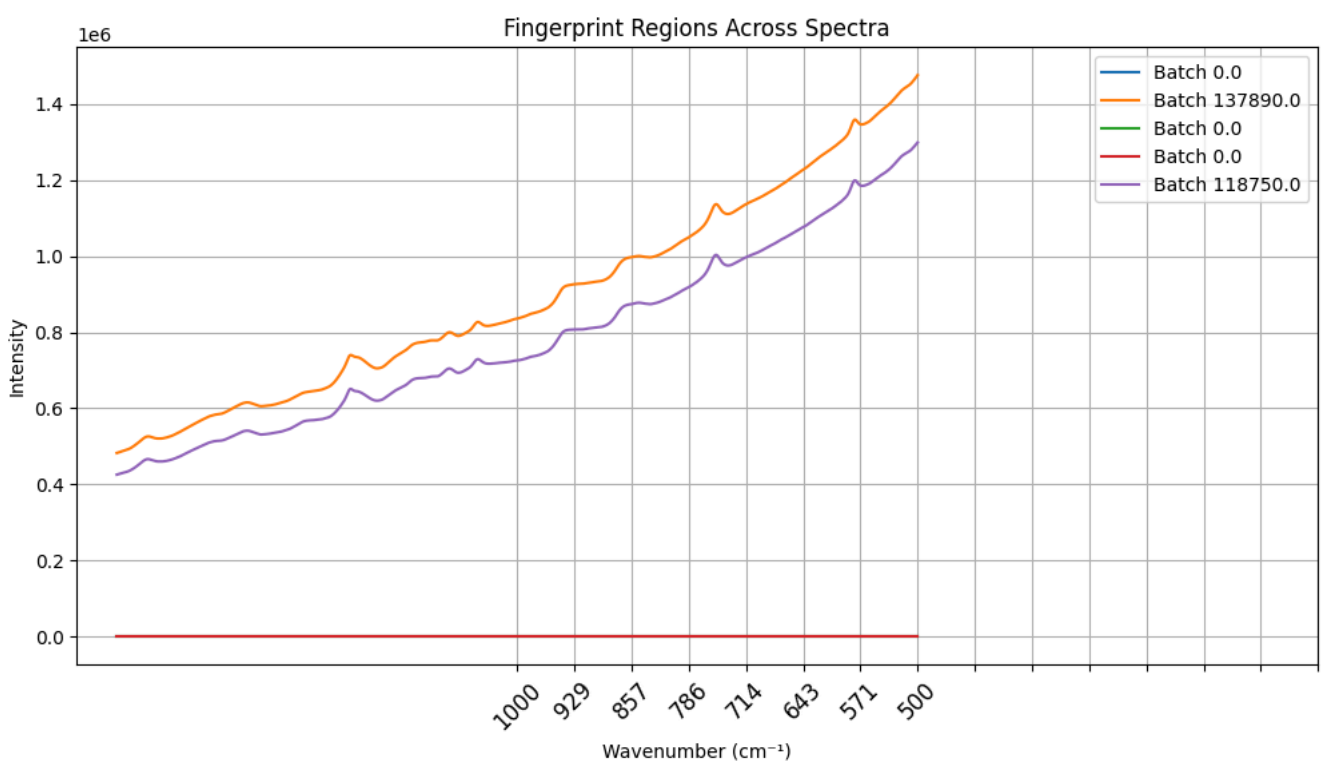
```
import pandas as pd

print("Dataset Columns:\n", df.columns)

literature_columns = [col for col in df.columns if 'literature' in col.lower() or 'source' in col.lower()]

literature_data = df[literature_columns]
print("\nExtracted Literature References:")
```

```
print(literature_data.head())
```

```
Dataset Columns:
 Index(['Time (h)', 'Aeration rate(Fg:L/h)', 'Agitator RPM(RPM:RPM)',
       'Sugar feed rate(Fs:L/h)', 'Acid flow rate(Fa:L/h)',
       'Base flow rate(Fb:L/h)', 'Heating/cooling water flow rate(Fc:L/h)',
       'Heating water flow rate(Fh:L/h)',
       'Water for injection/dilution(Fw:L/h)',
       'Air head pressure(pressure:bar)',
       ...
       '209', '208', '207', '206', '205', '204', '203', '202', '201',
       'Anomaly_Score'],
      dtype='object', length=2240)

Extracted Literature References:
Empty DataFrame
Columns: []
Index: [66548, 43476, 74523, 109263, 36238]
```

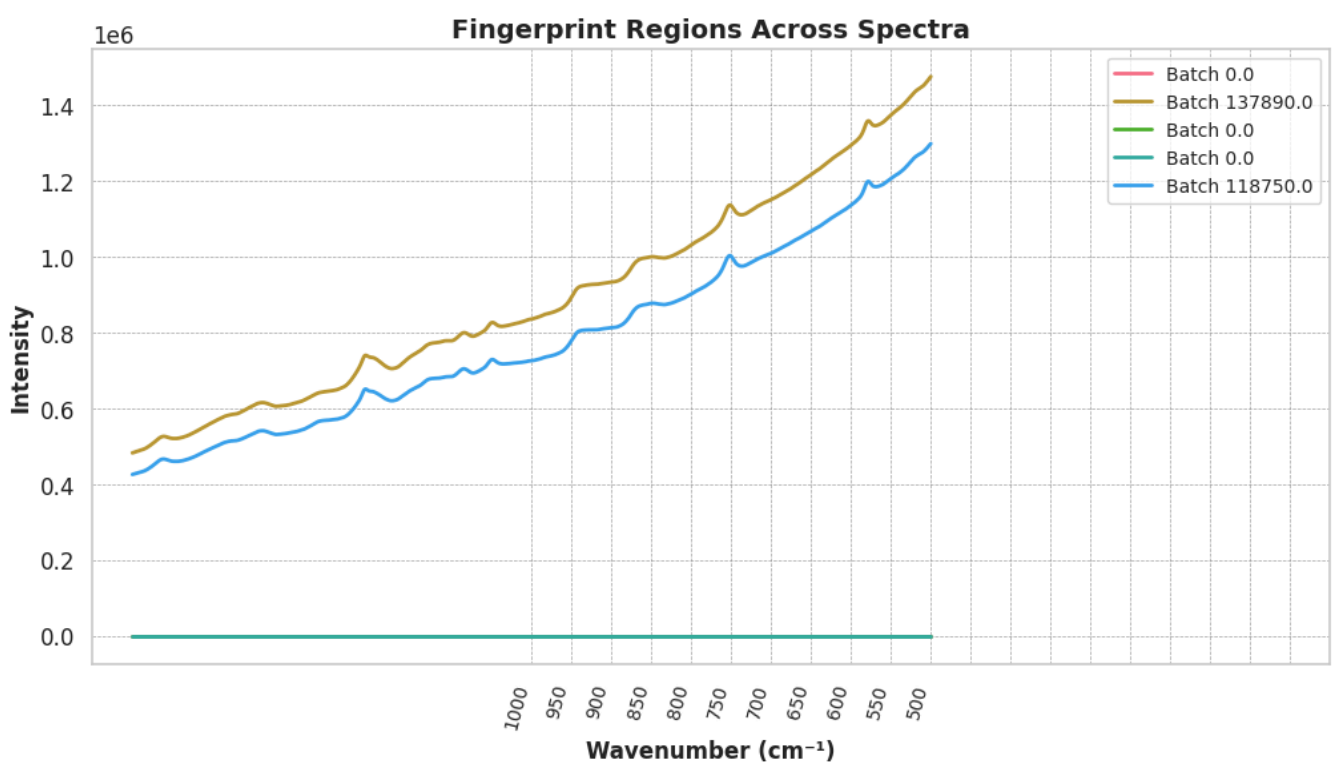
Fingerprint Range of Each Spectra

```
fingerprint_start, fingerprint_end = 500, 1500

spectral_columns = [col for col in df.columns if col.replace('.', '').isdigit()]
wn = np.array([float(col) for col in spectral_columns])

fingerprint_columns = [col for col in spectral_columns
                       if fingerprint_start <= float(col) <= fingerprint_end]

fingerprint_spectra = df[fingerprint_columns]

plt.figure(figsize=(12, 6))
for i in range(min(5, len(fingerprint_spectra))):
    plt.plot(fingerprint_columns, fingerprint_spectra.iloc[i], label=f'Batch {df["Batch ID"].iloc[i]}')
plt.xticks(ticks=np.linspace(fingerprint_start, fingerprint_end, num=15), rotation=45, fontsize=12)


plt.xlabel("Wavenumber (cm⁻¹)")
plt.ylabel("Intensity")
plt.title("Fingerprint Regions Across Spectra")
plt.legend()
plt.grid(True)
plt.show()
```



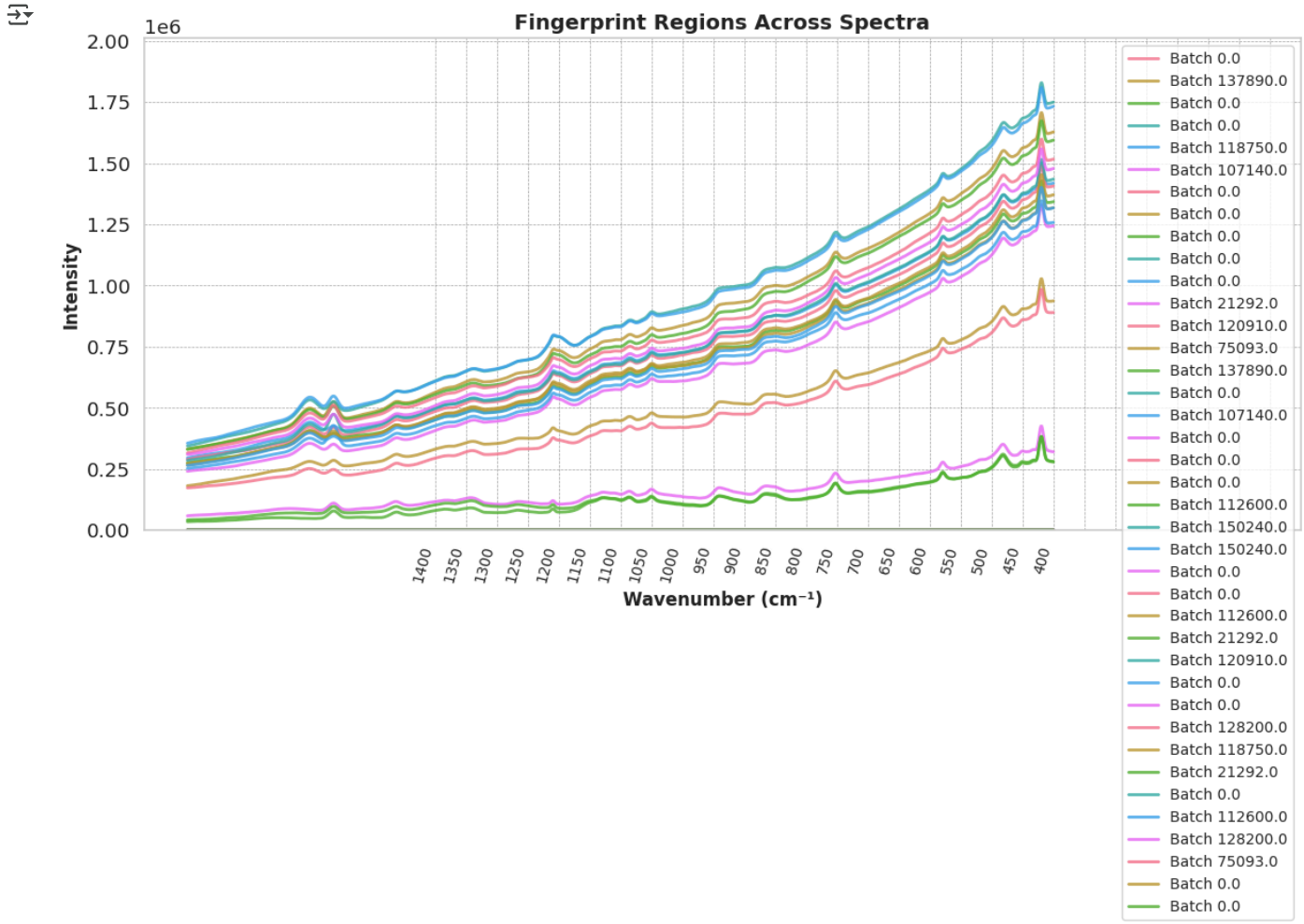

```
sns.set(style="whitegrid", palette="husl", font_scale=1.1)

plt.figure(figsize=(12, 6))
```

```
# Plot fingerprint spectra with smooth curves
for i in range(min(5, len(fingerprint_spectra))):
    plt.plot(fingerprint_columns, fingerprint_spectra.iloc[i], label=f'Batch {df["Batch ID"].iloc[i]}', linewidth=2)

# **Increase number of x-axis labels**
plt.xticks(ticks=np.arange(fingerprint_start, fingerprint_end+1, step=50),
           rotation=75, ha='right', fontsize=10)

# Labels & title formatting
plt.xlabel("Wavenumber (cm⁻¹)", fontsize=12, fontweight='bold')
plt.ylabel("Intensity", fontsize=12, fontweight='bold')
plt.title("Fingerprint Regions Across Spectra", fontsize=14, fontweight='bold')

# Styled legend placement
plt.legend(loc="upper right", fontsize=10, frameon=True)

# Subtle yet effective grid styling
plt.grid(color='gray', linestyle='dashed', linewidth=0.5, alpha=0.7)

plt.show()
```



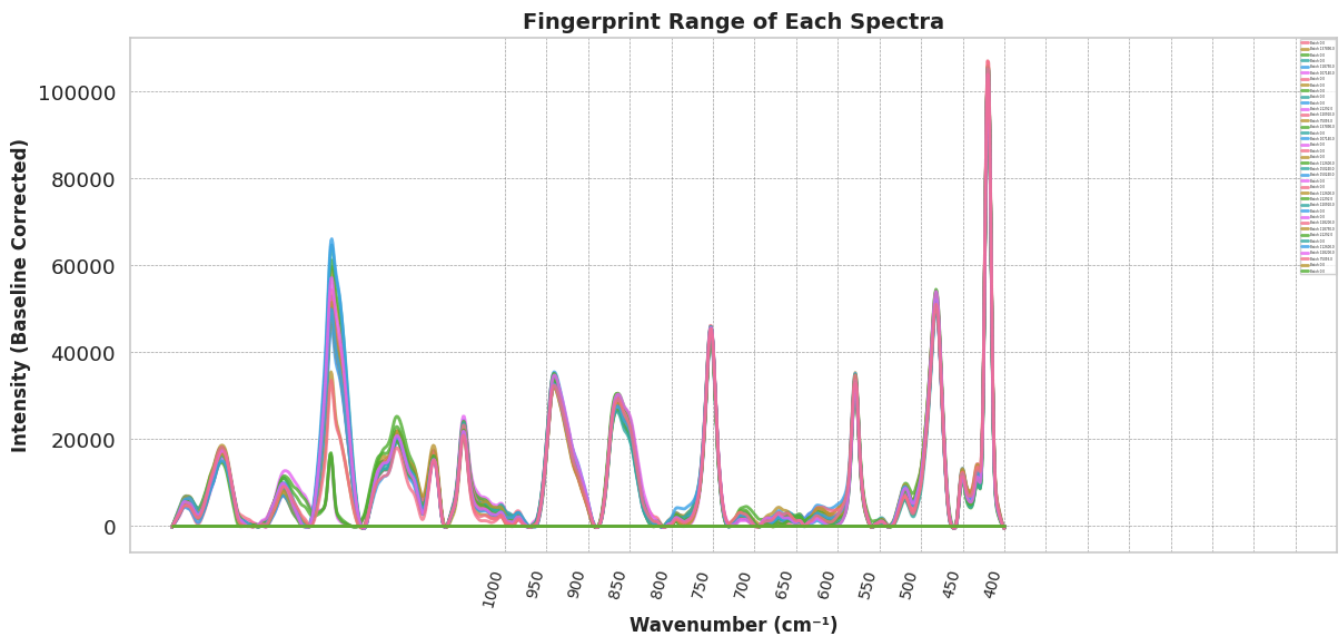

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Set a clean & polished style using Seaborn
sns.set(style="whitegrid", palette="husl", font_scale=1.2)

# Define fingerprint boundaries
fingerprint_start, fingerprint_end = 400, 1800

# Extract spectral columns
spectral_columns = [col for col in df.columns if col.replace('.', '').isdigit()]
fingerprint_columns = [col for col in spectral_columns if fingerprint_start <= float(col) <= fingerprint_end]

# Extract fingerprint spectra
fingerprint_spectra = df[fingerprint_columns]

# Initialize the figure with a larger size for better readability
plt.figure(figsize=(14, 6))

# Plot fingerprint spectra with smooth curves & balanced styling
for i in range(len(fingerprint_spectra)):
    plt.plot(fingerprint_columns, fingerprint_spectra.iloc[i],
             label=f'Batch {df["Batch ID"].iloc[i]}', linewidth=2, alpha=0.8)

# Improve x-axis ticks (more labels but readable)
plt.xticks(ticks=np.arange(fingerprint_start, fingerprint_end + 1, step=50),
           rotation=75, ha='right', fontsize=10)
```

```
# Ensure y-axis scaling prevents missing lines
plt.ylim([0, np.max(fingerprint_spectra.values.flatten()) * 1.1])

# Labels and title formatting for clarity
plt.xlabel("Wavenumber (cm⁻¹)", fontsize=12, fontweight='bold')
plt.ylabel("Intensity", fontsize=12, fontweight='bold')
plt.title("Fingerprint Regions Across Spectra", fontsize=14, fontweight='bold')

# Clean, compact legend
plt.legend(loc="upper right", fontsize=10, frameon=True)

# Grid styling for a polished look
plt.grid(color='gray', linestyle='dashed', linewidth=0.5, alpha=0.7)

# Show plot
plt.show()
```




```
# Define fingerprint boundaries
fingerprint_start, fingerprint_end = 400, 1400

# Extract spectral columns
spectral_columns = [col for col in df.columns if col.replace('.', '').isdigit()]
fingerprint_columns = [col for col in spectral_columns if fingerprint_start <= float(col) <= fingerprint_end]

# Extract fingerprint spectra
fingerprint_spectra = df[fingerprint_columns]

# Apply baseline correction for flattened spectra
baseline_fitter = Baseline(np.array(fingerprint_columns))
flattened_spectra = np.zeros_like(fingerprint_spectra.values)

for i in range(len(fingerprint_spectra)):
    baseline, _ = baseline_fitter.asls(fingerprint_spectra.iloc[i].values, lam=1e3, p=0.001)
    flattened_spectra[i] = fingerprint_spectra.iloc[i].values - baseline
```

```
# Smooth data for better peak visualization
smooth_spectra = savgol_filter(flattened_spectra, window_length=7, polyorder=2, axis=1)

# Plot fingerprint spectra with flattened baseline
plt.figure(figsize=(14, 6))
for i in range(len(fingerprint_spectra)):
    plt.plot(fingerprint_columns, smooth_spectra[i], label=f"Batch {df['Batch ID'].iloc[i]}", linewidth=2, alpha=0.8)

# Improve x-axis tick visibility
plt.xticks(ticks=np.arange(fingerprint_start, fingerprint_end + 1, step=50), rotation=75, ha='right', fontsize=10)

# Labels, title & legend
plt.xlabel("Wavenumber (cm⁻¹)", fontsize=12, fontweight='bold')
plt.ylabel("Intensity (Baseline Corrected)", fontsize=12, fontweight='bold')
plt.title("Fingerprint Range of Each Spectra", fontsize=14, fontweight='bold')
plt.legend(loc="upper right", fontsize=2, frameon=True)

# Grid styling for a polished look
plt.grid(color='gray', linestyle='dashed', linewidth=0.5, alpha=0.7)

plt.show()
```



Augment

```
!pip install shap
```

```
Collecting shap
  Downloading shap-0.47.2-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (25 kB)
Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages (from shap) (2.0.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.11/dist-packages (from shap) (1.15.3)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.11/dist-packages (from shap) (1.6.1)
Requirement already satisfied: pandas in /usr/local/lib/python3.11/dist-packages (from shap) (2.2.2)
Requirement already satisfied: tqdm>=4.27.0 in /usr/local/lib/python3.11/dist-packages (from shap) (4.67.1)
Requirement already satisfied: packaging>20.9 in /usr/local/lib/python3.11/dist-packages (from shap) (25.0)
Collecting slicer==0.0.8 (from shap)
  Downloading slicer-0.0.8-py3-none-any.whl.metadata (4.0 kB)
Requirement already satisfied: numba>=0.54 in /usr/local/lib/python3.11/dist-packages (from shap) (0.61.2)
Requirement already satisfied: cloudpickle in /usr/local/lib/python3.11/dist-packages (from shap) (3.1.1)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.11/dist-packages (from shap) (4.13.2)
Requirement already satisfied: llvmlite<0.45,>=0.44.0dev0 in /usr/local/lib/python3.11/dist-packages (from numba>=0.54->shap) (0.44
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.11/dist-packages (from pandas->shap) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-packages (from pandas->shap) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-packages (from pandas->shap) (2025.2)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn->shap) (1.5.0)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn->shap) (3.6.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (from python-dateutil>=2.8.2->pandas->shap) (1.17
Downloading shap-0.47.2-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (1.0 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.0/1.0 MB 15.1 MB/s eta 0:00:00
Downloading slicer-0.0.8-py3-none-any.whl (15 kB)
Installing collected packages: slicer, shap
Successfully installed shap-0.47.2 slicer-0.0.8
```

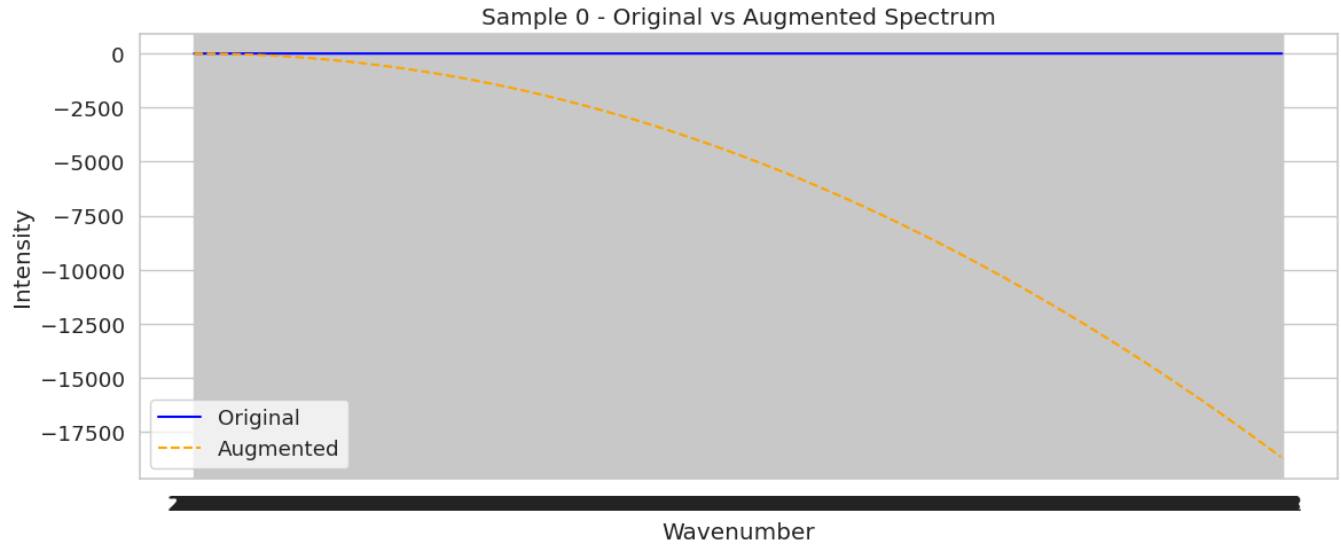
```
import numpy as np
import pandas as pd
import shap
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split


spectral_columns = [col for col in df.columns if col.replace('.', '').isdigit()]
spectra = df[spectral_columns].values
labels = df["Batch ID"].values


augmented_spectra = []
augmented_labels = []


for batch_id in np.unique(labels):
    batch_spectra = spectra[labels == batch_id]

    if len(batch_spectra) < 50:
        augment_multiplier = 20
    else:
        augment_multiplier = 10

    for spectrum in batch_spectra:
        for _ in range(augment_multiplier):
            noisy_spectrum = spectrum + np.random.normal(0, 0.05 * np.nanmax(spectrum), size=spectrum.shape)
            peak_shifted_spectrum = np.roll(noisy_spectrum, shift=np.random.randint(-5, 5))
            scaled_spectrum = peak_shifted_spectrum * np.random.uniform(0.9, 1.1)

            augmented_spectra.append(scaled_spectrum)
            augmented_labels.append(batch_id)


augmented_df = pd.DataFrame(augmented_spectra, columns=spectral_columns)
augmented_df["Batch ID"] = augmented_labels


X_train, X_test, y_train, y_test = train_test_split(augmented_df[spectral_columns], augmented_df["Batch ID"], test_size=0.2, random_stat
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
```

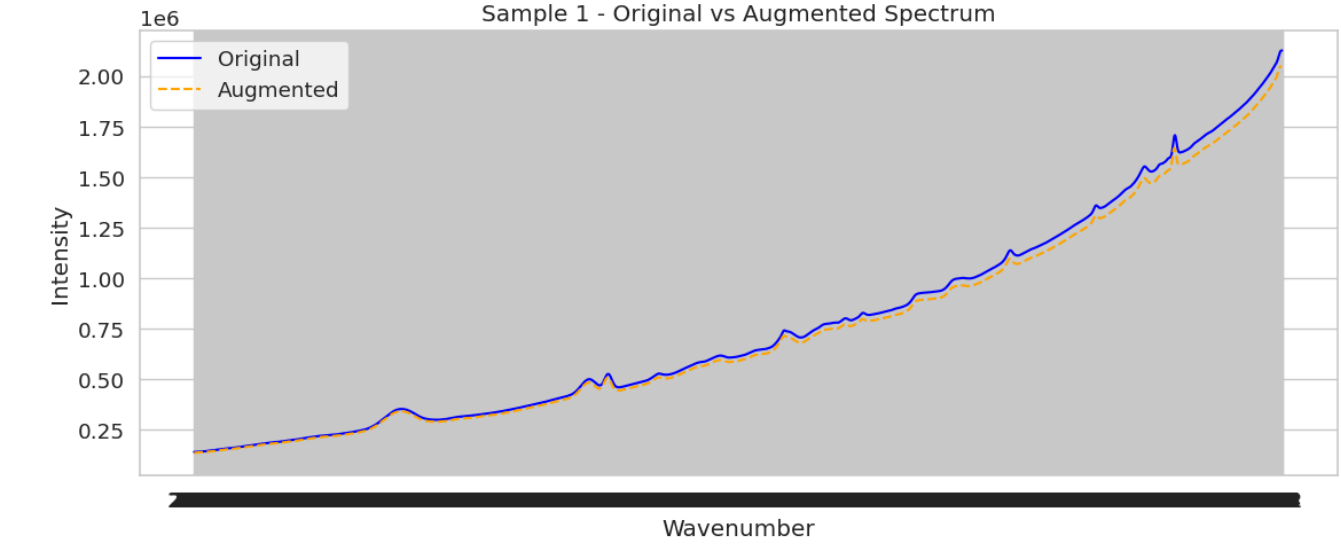
```
RandomForestClassifier
RandomForestClassifier(random_state=42)
```

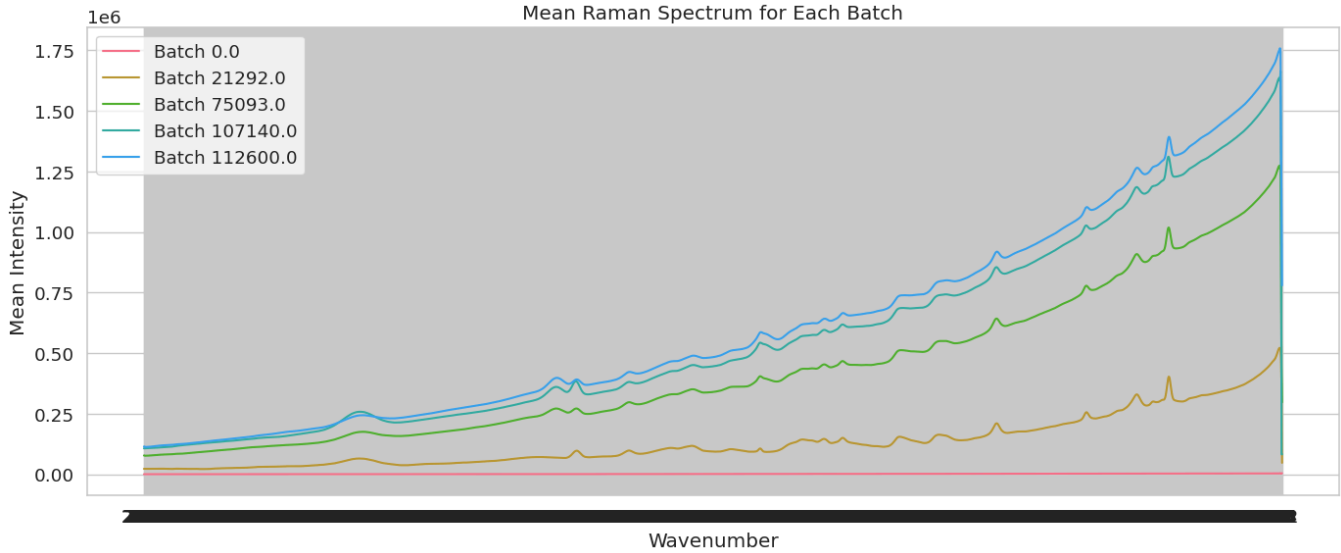
```
explainer = shap.TreeExplainer(model, feature_perturbation="auto")
shap_values = explainer.shap_values(X_test, check_additivity=False)


shap_values_corrected = shap_values[0] if isinstance(shap_values, list) else shap_values


import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# ==============================
# 🔁 Basic Augmentation Functions
# ==============================

def add_noise(spectrum, noise_level=0.01):
    return spectrum + np.random.normal(0, noise_level, size=spectrum.shape)

def shift_spectrum(spectrum, shift_range=(-3, 3)):
    shift = np.random.randint(*shift_range)
    return np.roll(spectrum, shift)

def scale_intensity(spectrum, factor_range=(0.9, 1.1)):
    factor = np.random.uniform(*factor_range)
    return spectrum * factor

def baseline_drift(spectrum):
    baseline = np.polyval(np.random.uniform(-0.01, 0.01, size=3), np.arange(len(spectrum)))
    return spectrum + baseline

# ==============================
# 🧠 Apply All Augmentations
# ==============================
```

```
def augment_spectrum(spectrum):
    spectrum = add_noise(spectrum)
    spectrum = shift_spectrum(spectrum)
    spectrum = scale_intensity(spectrum)
    spectrum = baseline_drift(spectrum)
    return spectrum

# ==============================
# 📈 Visualize Original vs Augmented
# ==============================

def plot_augmentation_effect(df, spectral_columns, n_samples=2):
    fig, axs = plt.subplots(n_samples, 1, figsize=(12, 5 * n_samples))
    if n_samples == 1:
        axs = [axs]

    for i in range(n_samples):
        original = df[spectral_columns].iloc[i].values
        augmented = augment_spectrum(original)
        axs[i].plot(spectral_columns, original, label='Original', color='blue')
        axs[i].plot(spectral_columns, augmented, label='Augmented', color='orange', linestyle='--')
        axs[i].set_title(f'Sample {i} - Original vs Augmented Spectrum')
        axs[i].set_xlabel('Wavenumber')
        axs[i].set_ylabel('Intensity')
        axs[i].legend()

    plt.tight_layout()
    plt.show()

# ==============================
# ♻️ Apply to Full DataFrame
# ==============================

def apply_augmentation(df, spectral_columns, n_augments=3):
    augmented = []
    for _ in range(n_augments):
        new_df = df.copy()
        new_df[spectral_columns] = df[spectral_columns].apply(lambda row: augment_spectrum(row.values), axis=1, result_type='expand')
        augmented.append(new_df)
    return pd.concat([df] + augmented, ignore_index=True)

# ==============================
# 💉 Batch-Level Features
# ==============================

def create_batch_features(df, spectral_columns, batch_col='Batch ID'):
    grouped = df.groupby(batch_col)
    batch_features = grouped[spectral_columns].agg(['mean', 'std', 'min', 'max'])
    batch_features.columns = ['_'.join(col) for col in batch_features.columns]
    batch_features.reset_index(inplace=True)
    return batch_features

# ==============================
# 📊 Plot Batch Mean Spectra
# ==============================

def plot_batch_means(df, spectral_columns, batch_col='Batch ID', n_batches=5):
    plt.figure(figsize=(14, 6))
    for i, (batch_id, group) in enumerate(df.groupby(batch_col)):
        if i >= n_batches:
            break
        mean_spectrum = group[spectral_columns].mean()
        plt.plot(spectral_columns, mean_spectrum, label=f'Batch {batch_id}')
    plt.title('Mean Raman Spectrum for Each Batch')
    plt.xlabel('Wavenumber')
    plt.ylabel('Mean Intensity')
    plt.legend()
    plt.tight_layout()
    plt.show()

# ==============================
# 🚀 Example Usage
# ==============================

# Define spectral feature columns dynamically from your dataset
spectral_columns = [col for col in df.columns if col.replace('.', '').isdigit()]

# Plot before augmentation
plot_augmentation_effect(df, spectral_columns, n_samples=2)

# Apply augmentations
aug_df = apply_augmentation(df, spectral_columns, n_augments=3)
```

```
# Plot batch-level mean spectra
plot_batch_means(aug_df, spectral_columns, batch_col='Batch ID', n_batches=5)

# Create batch-level aggregated features
batch_features_df = create_batch_features(aug_df, spectral_columns, batch_col='Batch ID')
print(batch_features_df.head())
```







	Batch ID	2400_mean	2400_std	2400_min	2400_max	\
0	0.0	0.001971	0.009566	-0.016641	0.029134	
1	21292.0	22310.804915	1125.789884	20701.460050	24218.246848	
2	75093.0	77272.176616	3071.849289	74795.000000	82667.669199	
3	107140.0	106674.016567	5910.486422	98725.895125	116727.233519	
4	112600.0	114186.218626	4103.759892	106423.565765	120181.878142	

	2399_mean	2399_std	2399_min	2399_max	2398_mean	\
0	0.000977	0.011453	-0.028231	0.033875	0.001919	
1	22253.833459	1287.437466	20291.172191	24706.559302	22439.376815	
2	77337.817898	3017.731073	75081.000000	82651.176093	76236.636474	
3	106772.672272	5904.961131	98845.084265	116738.064976	106786.584740	
4	114137.967998	3999.762286	106309.814976	120171.194318	111920.423071	

	...	203_min	203_max	202_mean	202_std	202_min	\
0	...	-46393.173657	4.302413e+04	2.823993e+03	25684.062669	-46435.430627	
1	...	68067.995588	5.500829e+05	3.565197e+05	250242.668809	68748.256677	